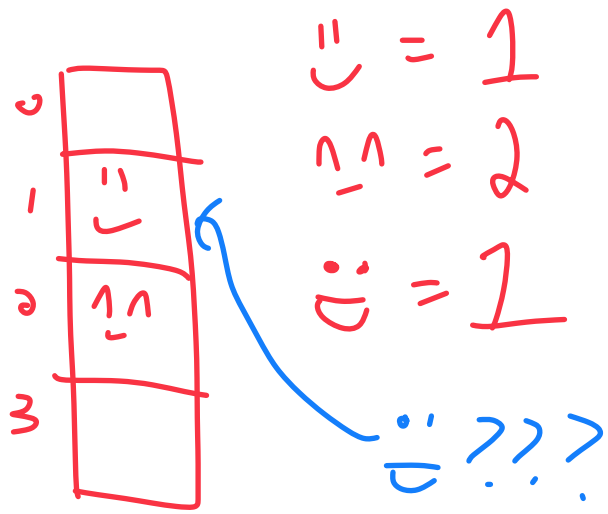
Algorithms and Data Structures for Data Science

Hashing 2

CS 277

Brad Solomon

March 5, 2025



UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

Department of Computer Science

Announcements

Fill out MP_generate feedback form for +2 EC (collectively)

Fill out IEF feedback form for +4 EC (collectively)

Exam 1 next week — practice exam out now!

Exam content can be found on the website (Exams Page)

Next Monday come prepared with questions (Exam 1 Review)

Learning Objectives

Review fundamentals of hash tables

Discuss what a 'good' hash function looks like

Identify a key weakness of the hash table

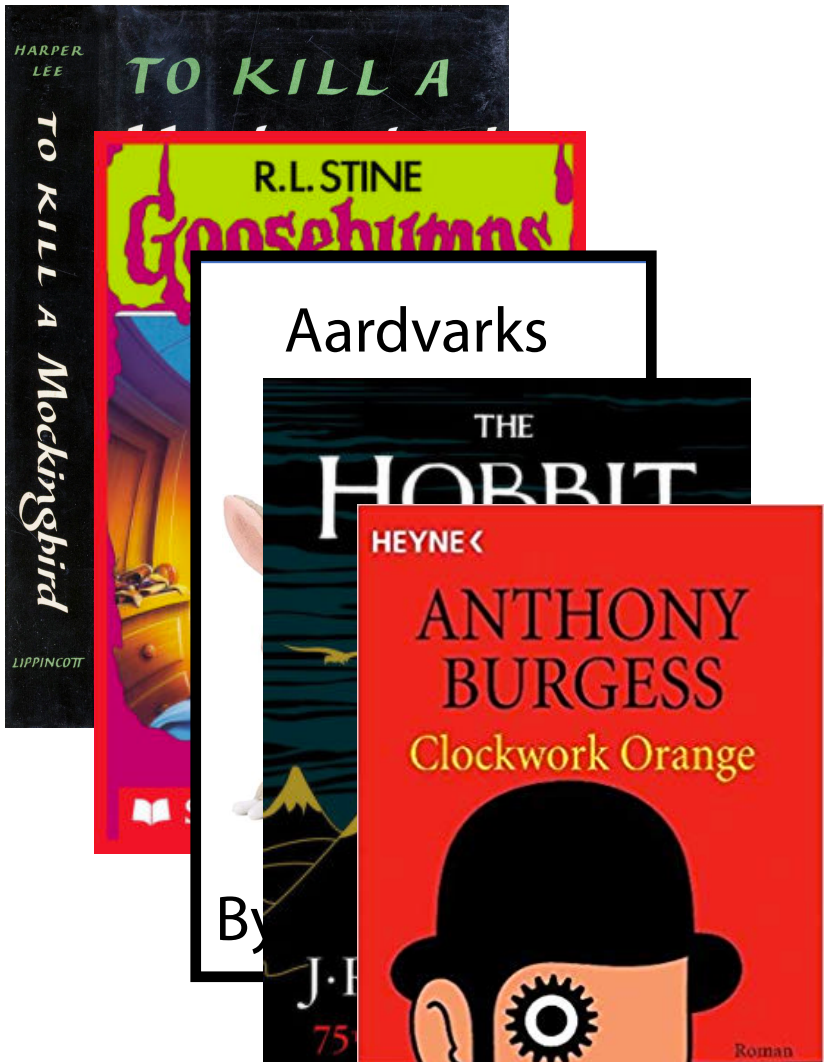
Introduce strategies to address this weakness

Introduce closed hashing approaches to hash collisions

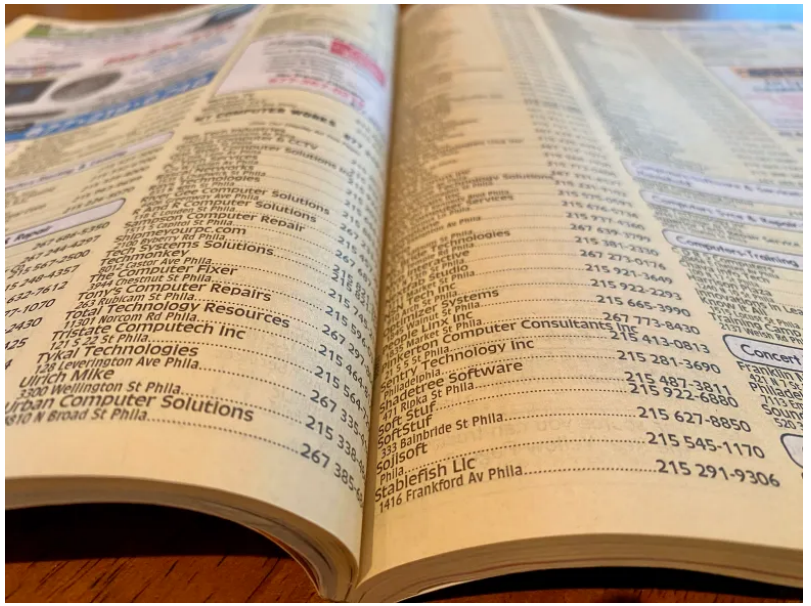
Determine when and how to resize a hash table

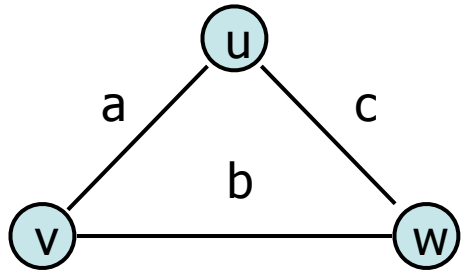
Key, Value Paired Data

Data as key, value pairs is an extremely common format in data science




```
Brad      CS 173, CS 225, CS 277
Geoffrey  CS 125, CS 233
Carl      CS 173, CS 225, CS 340
Wade      CS 107, CS 225, CS 340
```






```
u   (a,v), (c,w)
v   (a,y), (b,w)
w   (c,u), (b,v)
```

Dictionary ADT

A dictionary is an **unordered** collection of (key, value) pairs

The keys can be of any ***hashable*** (immutable) type

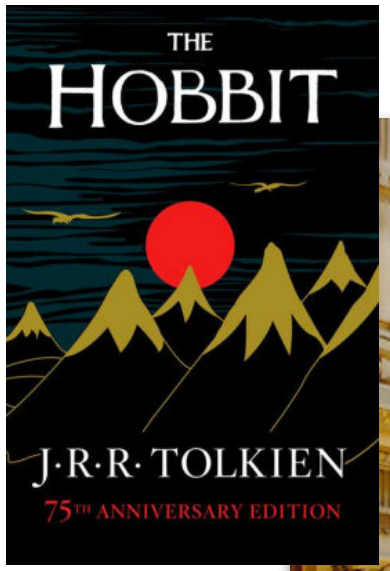
The values can be of any type

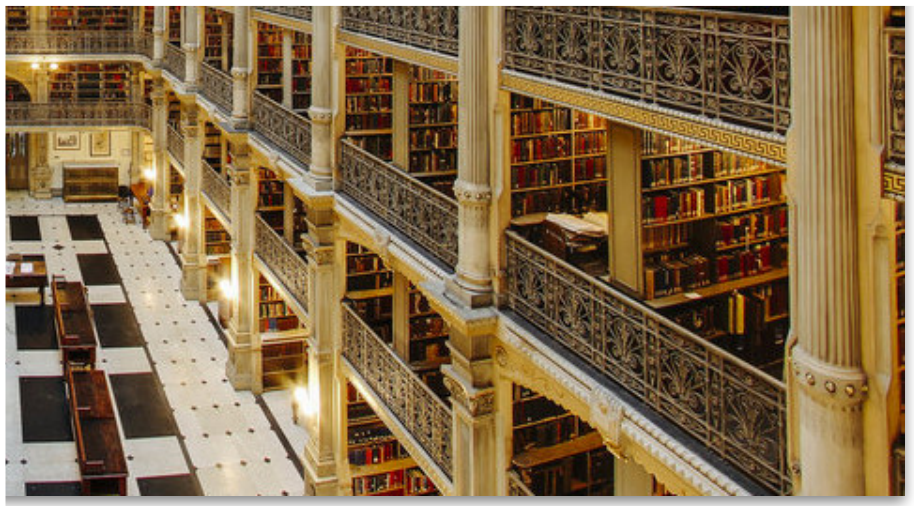
A minimal set of operations (that all dictionaries should have):

1. **Insert:** Add a new (key, value) pair to the dictionary
2. **Delete:** Remove a specific (key, value) pair from the dictionary
3. **getKeys:** Return the set of keys stored in the dictionary
4. **getValue:** Given a key, get the corresponding value
5. **Constructor:** Create a new empty dictionary

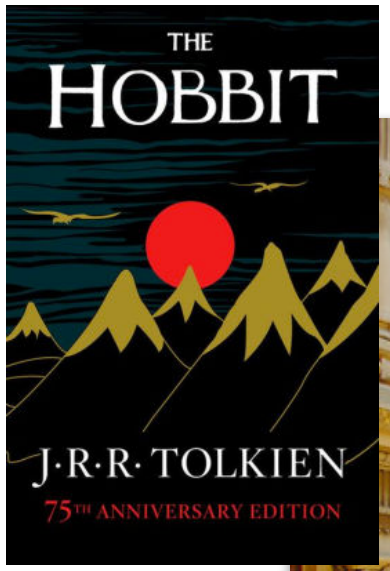
access

If we recognize that libraries are ordered: $O(log\ n)$





The library ‘hash table’

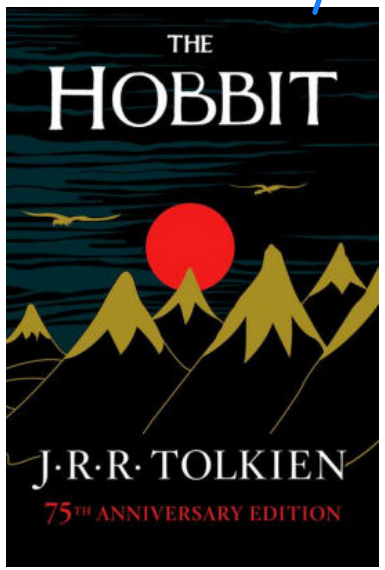


The library 'hash table'

Key

Hash function

Hash value address



O(1)

O(1)

ISBN: 9781526602381

Call #: PR
6068.O93
H35 1998

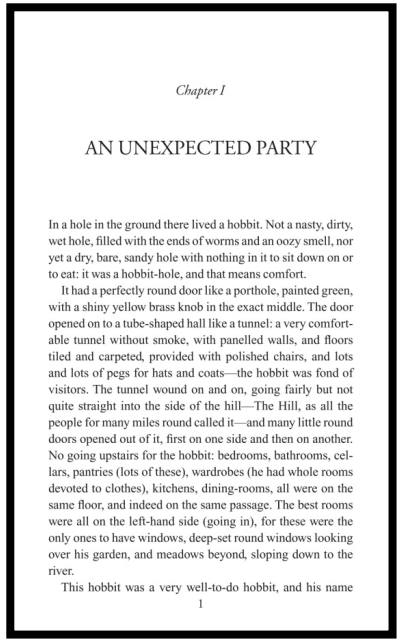
ISBN: 9781526602381

Call #: PR
6068.O93
H35 1998

O(1)

O(1)

Value



Chapter I

AN UNEXPECTED PARTY

In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.

It had a perfectly round door like a porthole, painted green, with a shiny yellow brass knob in the exact middle. The door opened on to a tube-shaped hall like a tunnel: a very comfortable tunnel without smoke, with panelled walls, and floors tiled and carpeted, provided with polished chairs, and lots and lots of pegs for hats and coats—the hobbit was fond of visitors. The tunnel wound on and on, going fairly but not quite straight into the side of the hill—The Hill, as all the people for many miles round called it—and many little round doors opened out of it, first on one side and then on another. No going upstairs for the hobbit: bedrooms, bathrooms, cellars, pantries (lots of these), wardrobes (he had whole rooms devoted to clothes), kitchens, dining-rooms, all were on the same floor, and indeed on the same passage. The best rooms were all on the left-hand side (going in), for these were the only ones to have windows, deep-set round windows looking over his garden, and meadows beyond, sloping down to the river.

This hobbit was a very well-to-do hobbit, and his name

1

A Hash Table based Dictionary

```
1 d = {}
2 d[k] = v
3 print(d[k])
```

A **Hash Table** consists of three things:

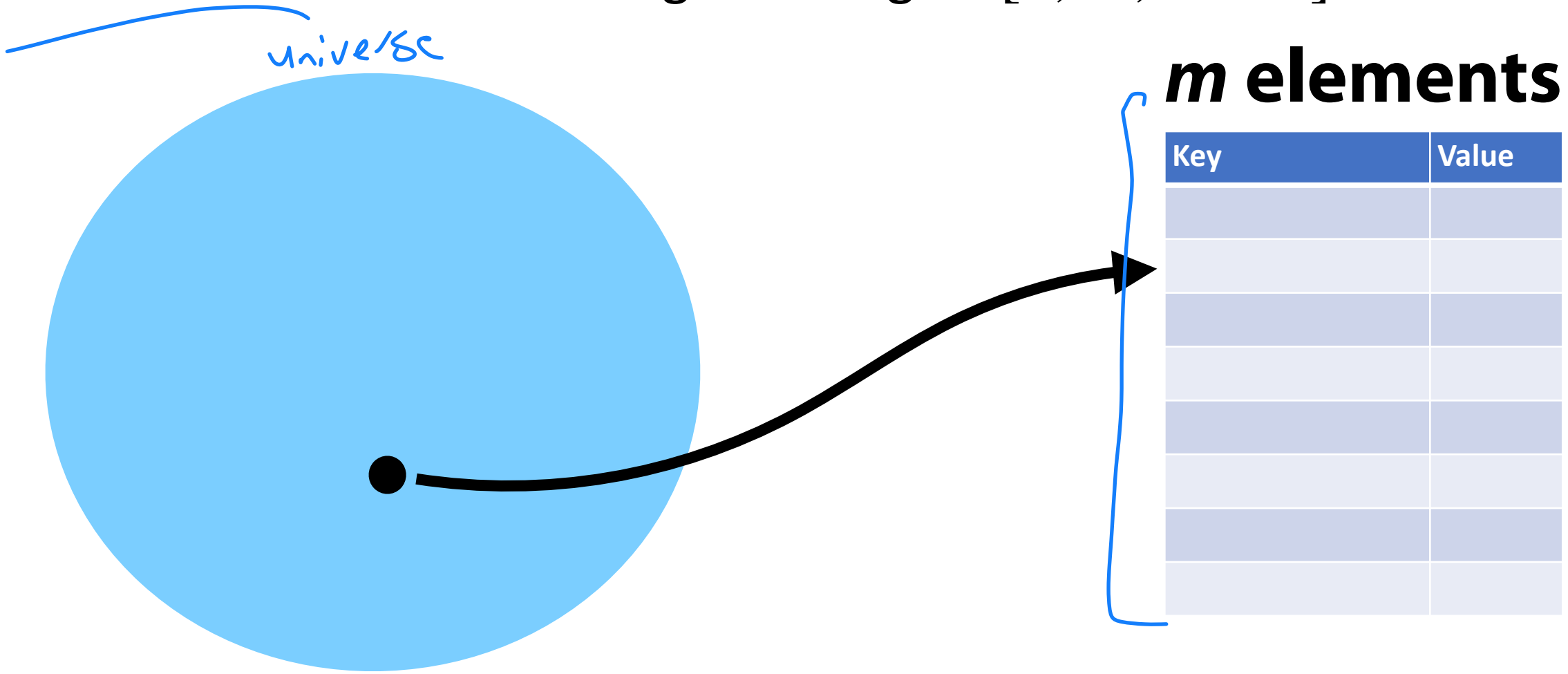
1. A hash function

2. A data storage structure (Array)

3. ???

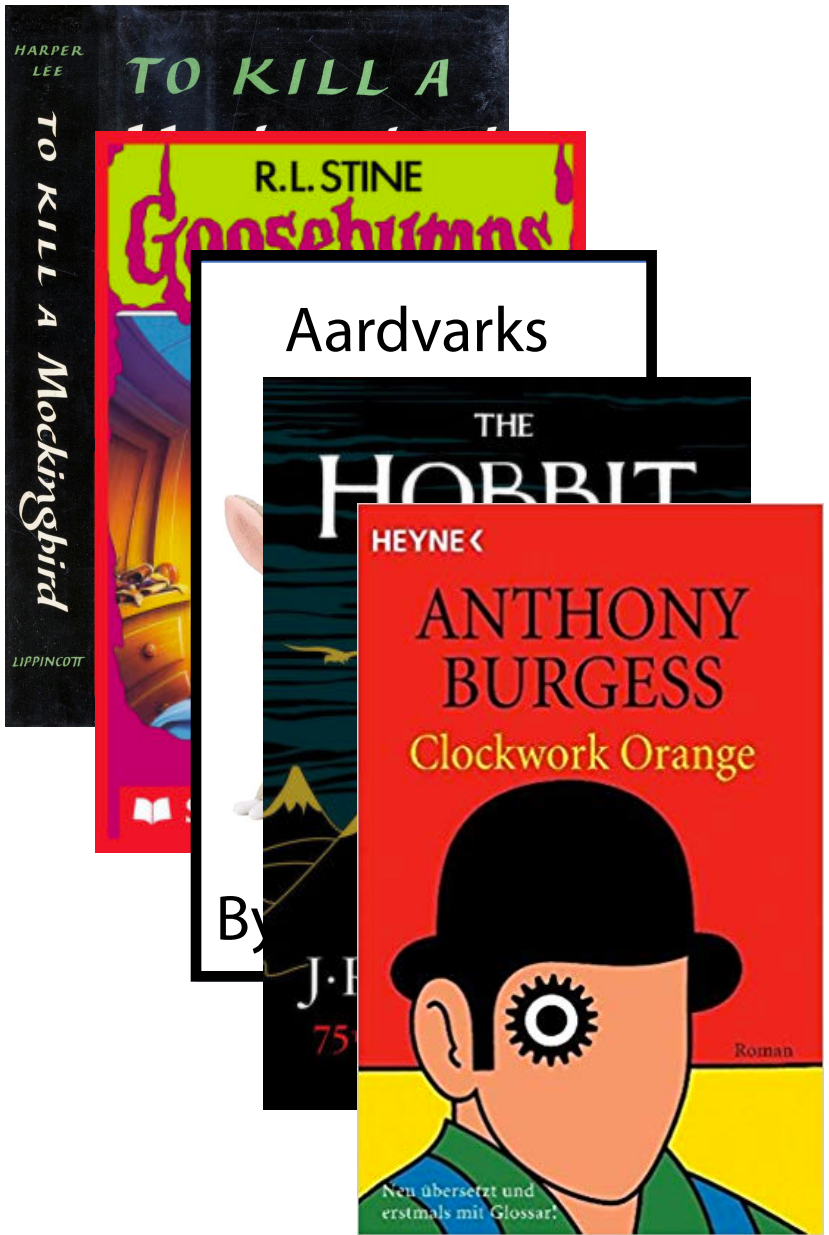
Hash Function

An $O(1)$ deterministic operation that maps all keys in a universe U to a defined range of integers $[0, \ldots, m-1]$



Hash Function

What are some hash functions for books?



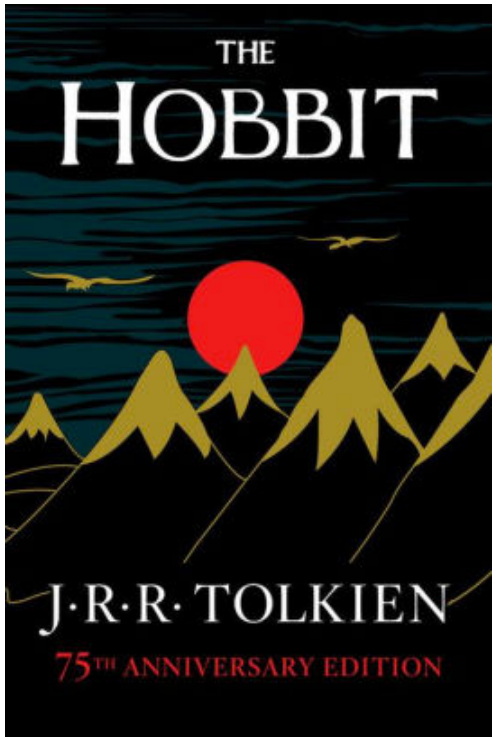
↳ Author Name

↳ Page #

↳ Mixed title letters

What defines a **good** hash function?

A Hash Table based Dictionary

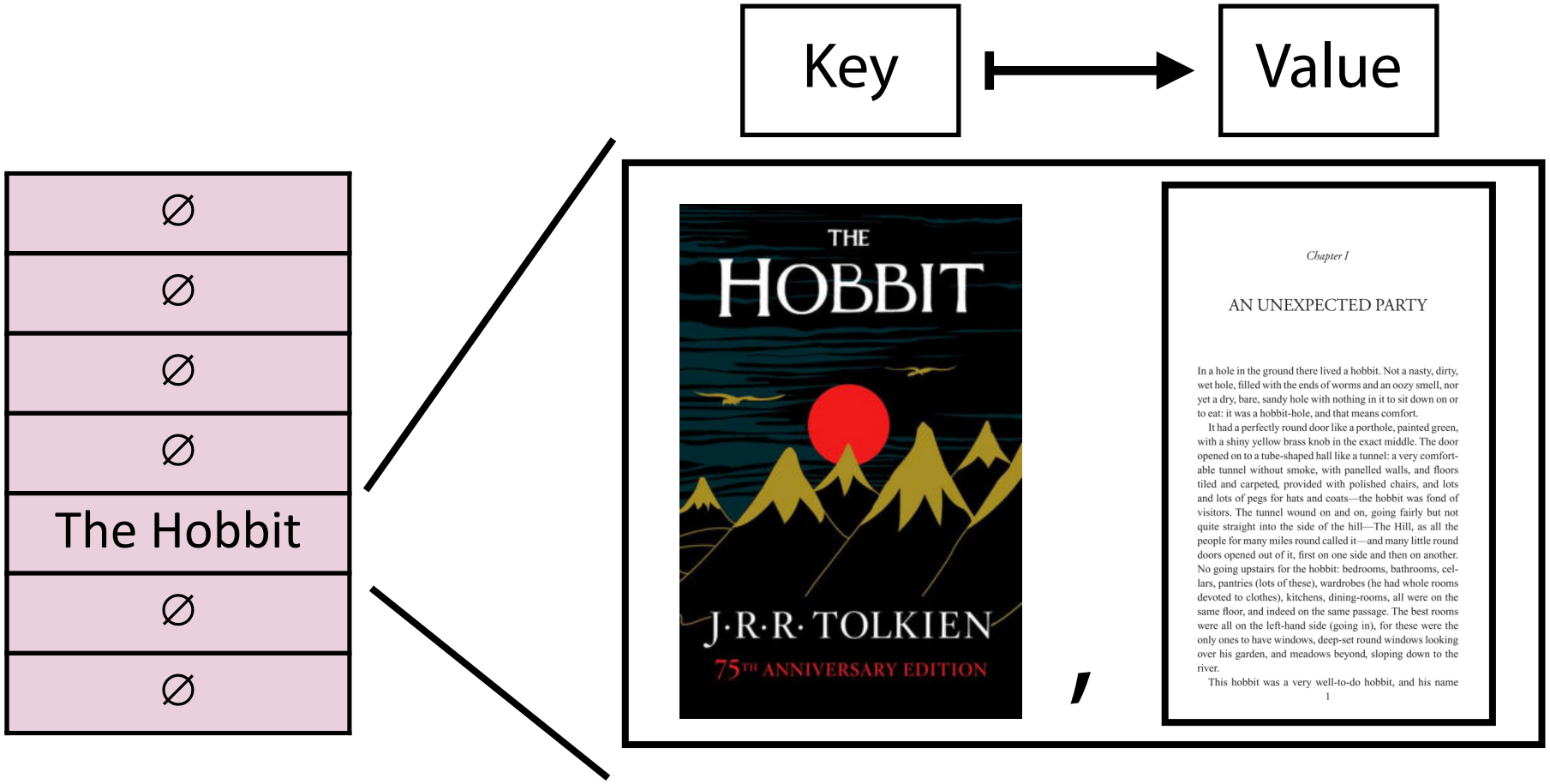


'J' + 'T' = 30

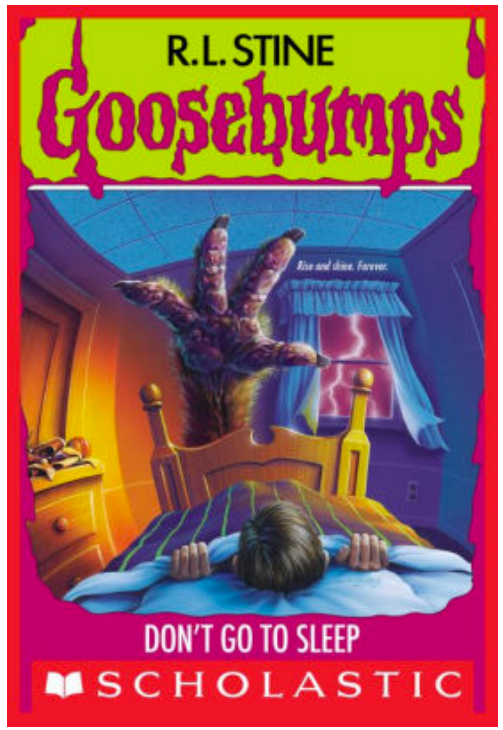
Author Name Hash Function

...	...
27	∅
28	∅
29	∅
30	The Hobbit
31	∅
...	...

A Hash Table based Dictionary



A Hash Table based Dictionary

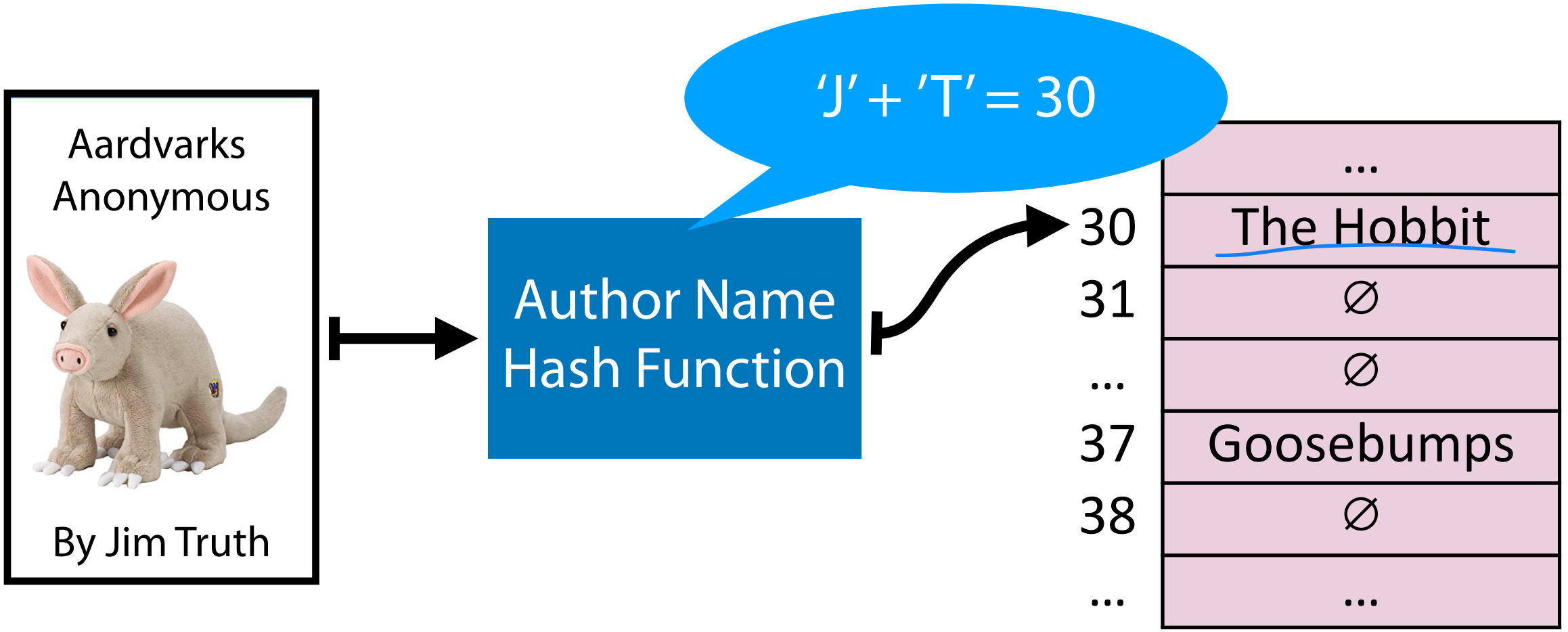


'R' + 'S' = 37

Author Name Hash Function

…	
30	The Hobbit
31	∅
…	∅
37	Goosebumps
38	∅
…	…

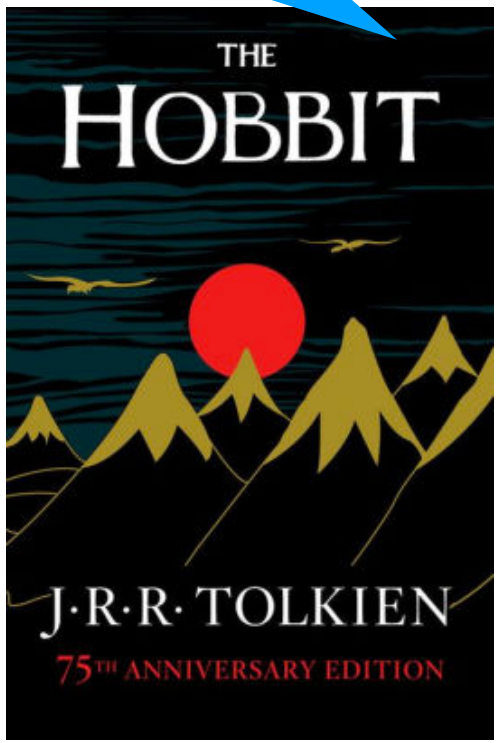
A Hash Table based Dictionary



Hash Collision

A ***hash collision*** occurs when multiple unique keys hash to the same value

J.R.R Tolkien = 30!



Jim Truth = 30!

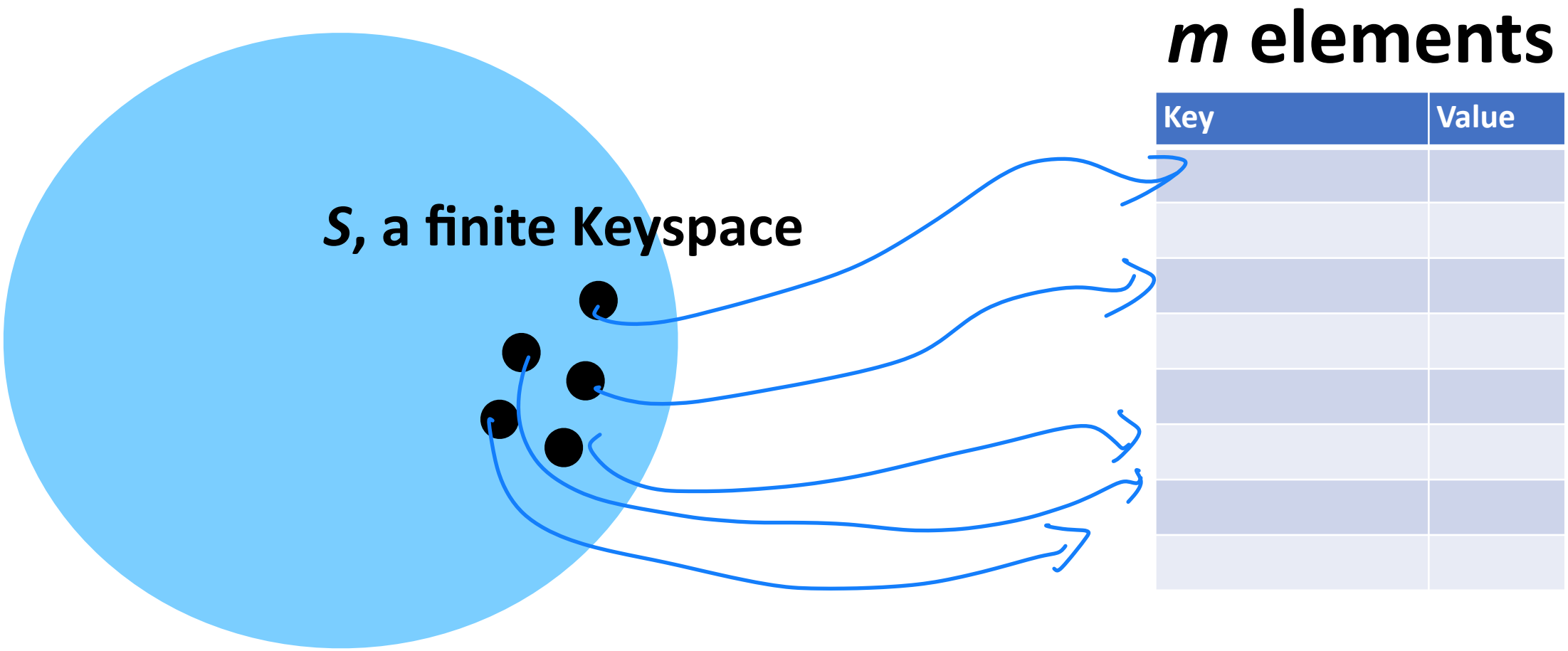
Aardvarks Anonymous

By Jim Truth

…	…
30	???
31	∅
…	∅
37	Goosebumps
38	∅
…	…

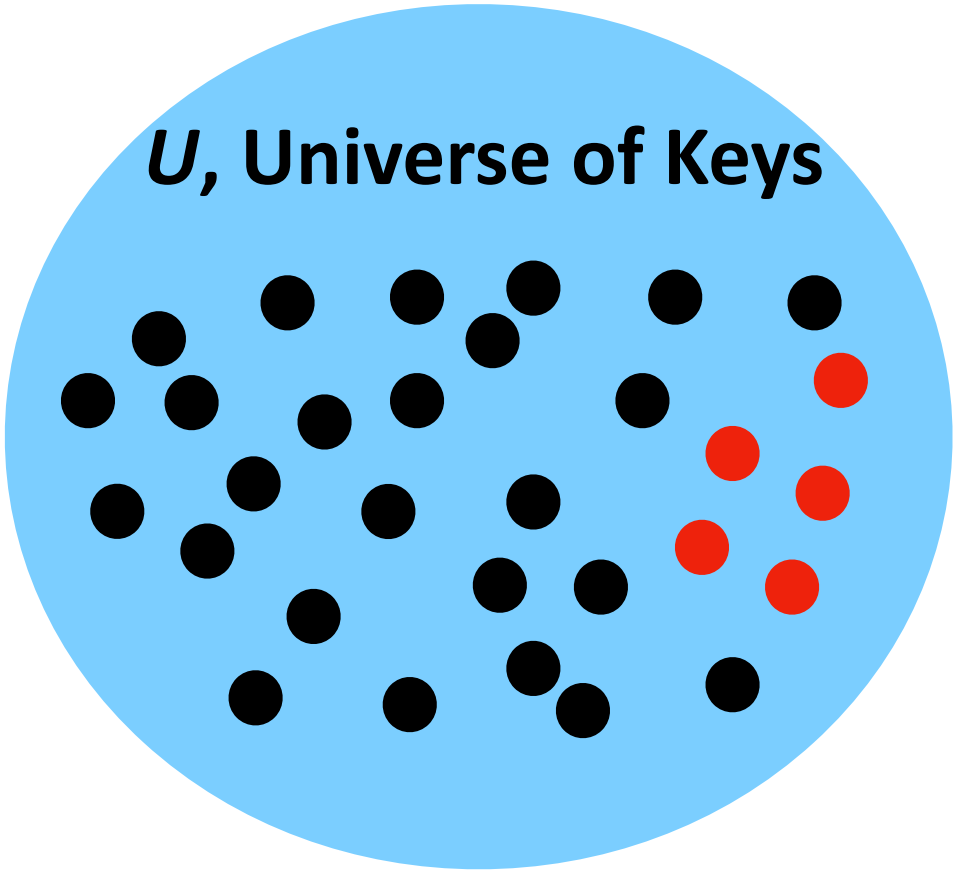
Perfect Hashing

If $m \geq S$, we can write a *perfect* hash with no collisions



General Purpose Hashing

In CS 277, we want our hash functions to work *in general*.



***m* elements**

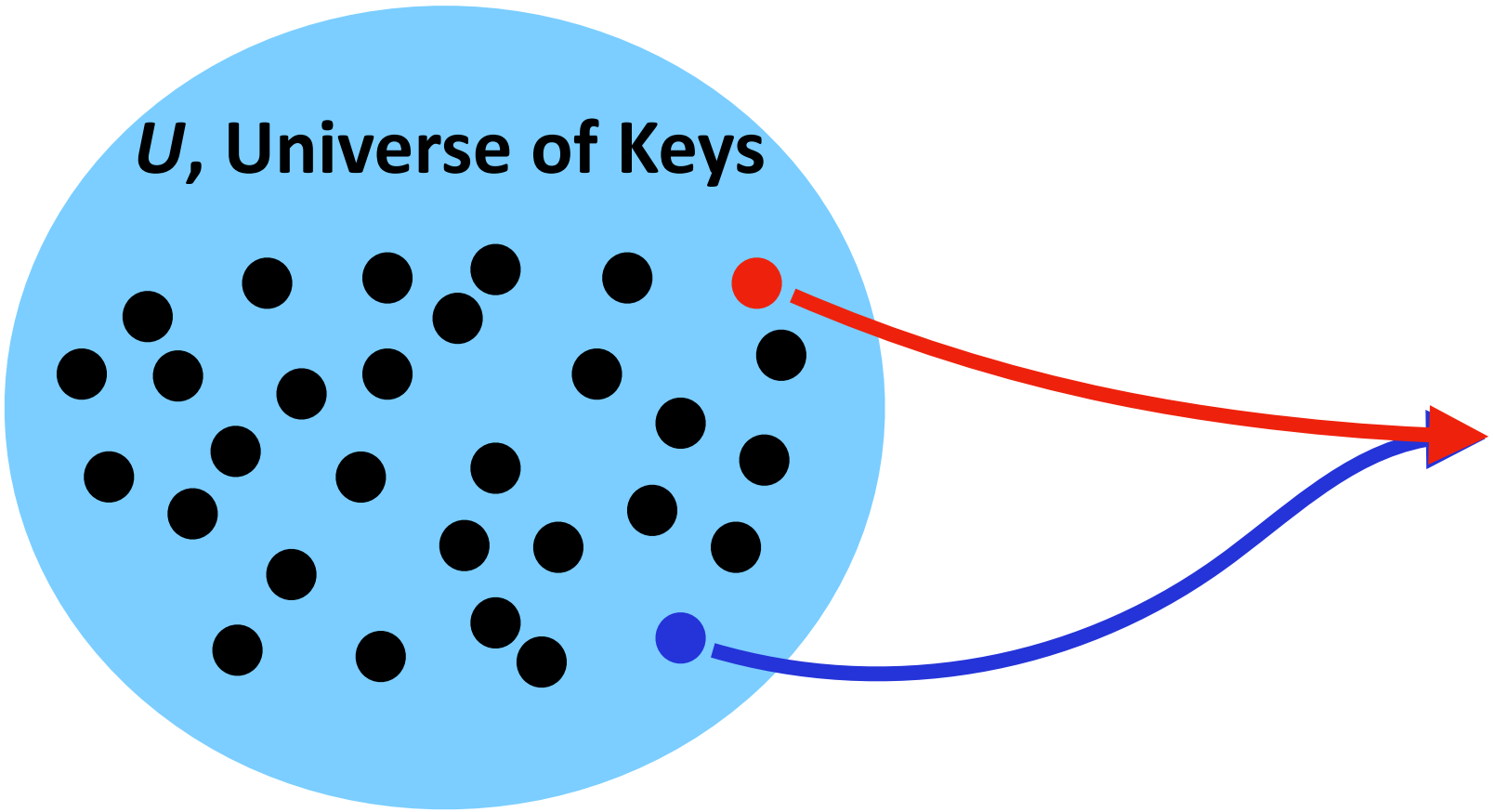
Key	Value

General Purpose Hashing

If $m < U$, there must be at least one hash collision.

Book titles

100



m elements

Key	Value

General Purpose Hashing

By fixing h, we open ourselves up to adversarial attacks.

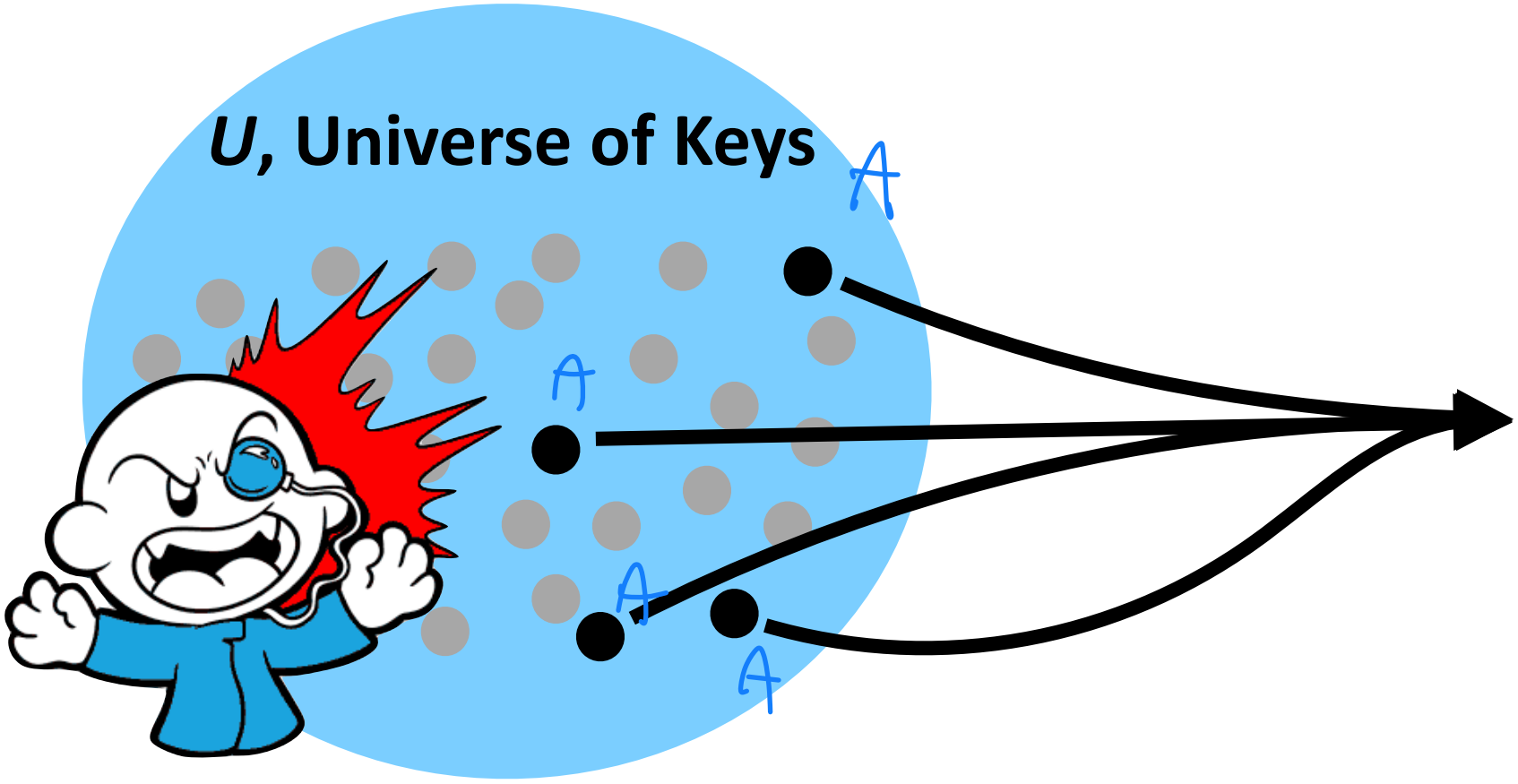


Image by Matthew Loffhagen

***m* elements**

Key	Value

A Hash Table based Dictionary

```
1 d = {}
2 d[k] = v
3 print(d[k])
```

A **Hash Table** consists of three things:

1. A hash function

A good hash function minimizes collision frequency

2. A data storage structure

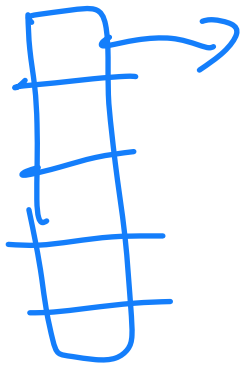
3. A method of addressing *hash collisions*

A good hash table resolves collisions efficiently

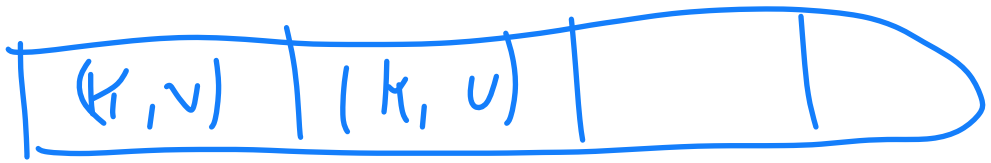
Open vs Closed Hashing

Addressing hash collisions depends on your storage structure.

- **Open Hashing:** Store (k, v) outside the hash table

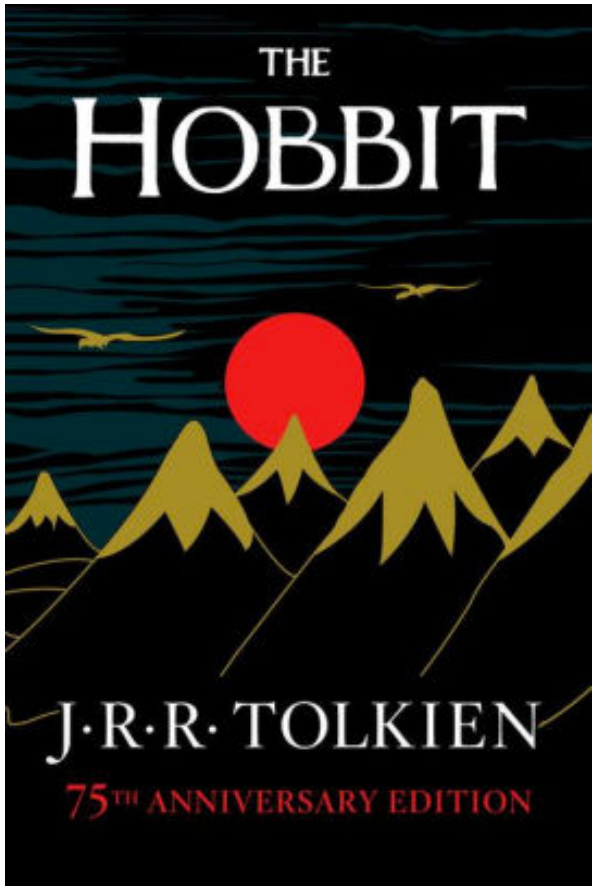


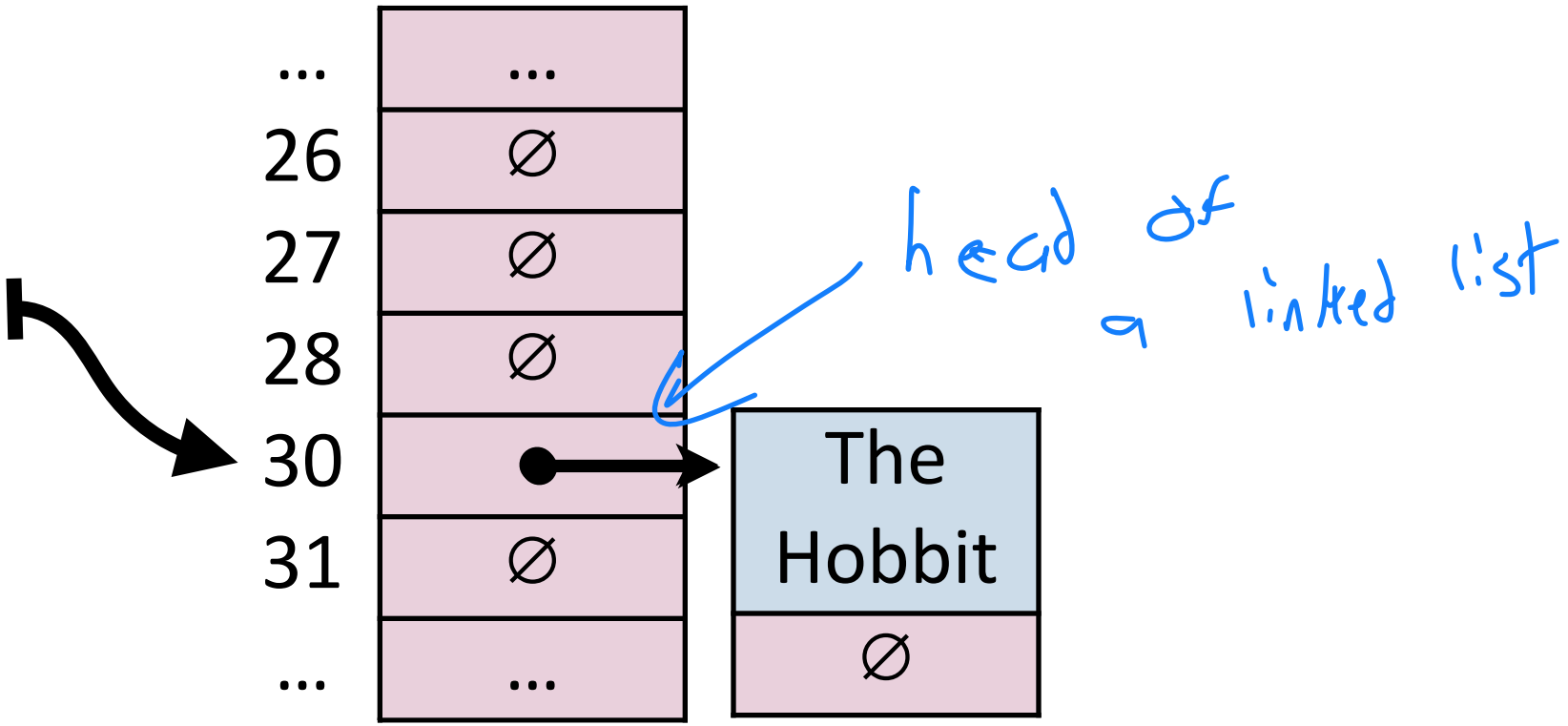
- **Closed Hashing:** Stores (k, v) inside the table



Open Hashing

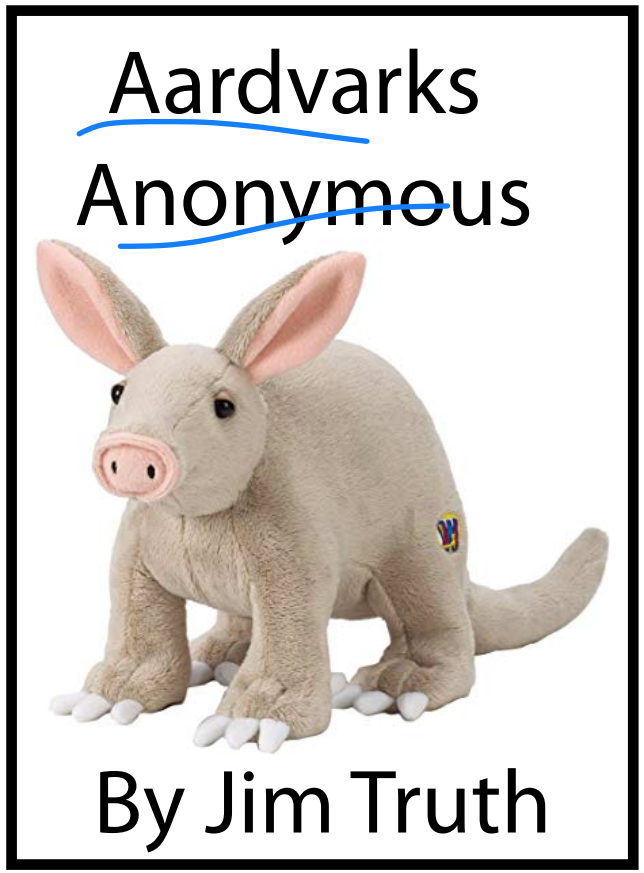
In an ***open hashing*** scheme, key-value pairs are stored externally (for example as a linked list).

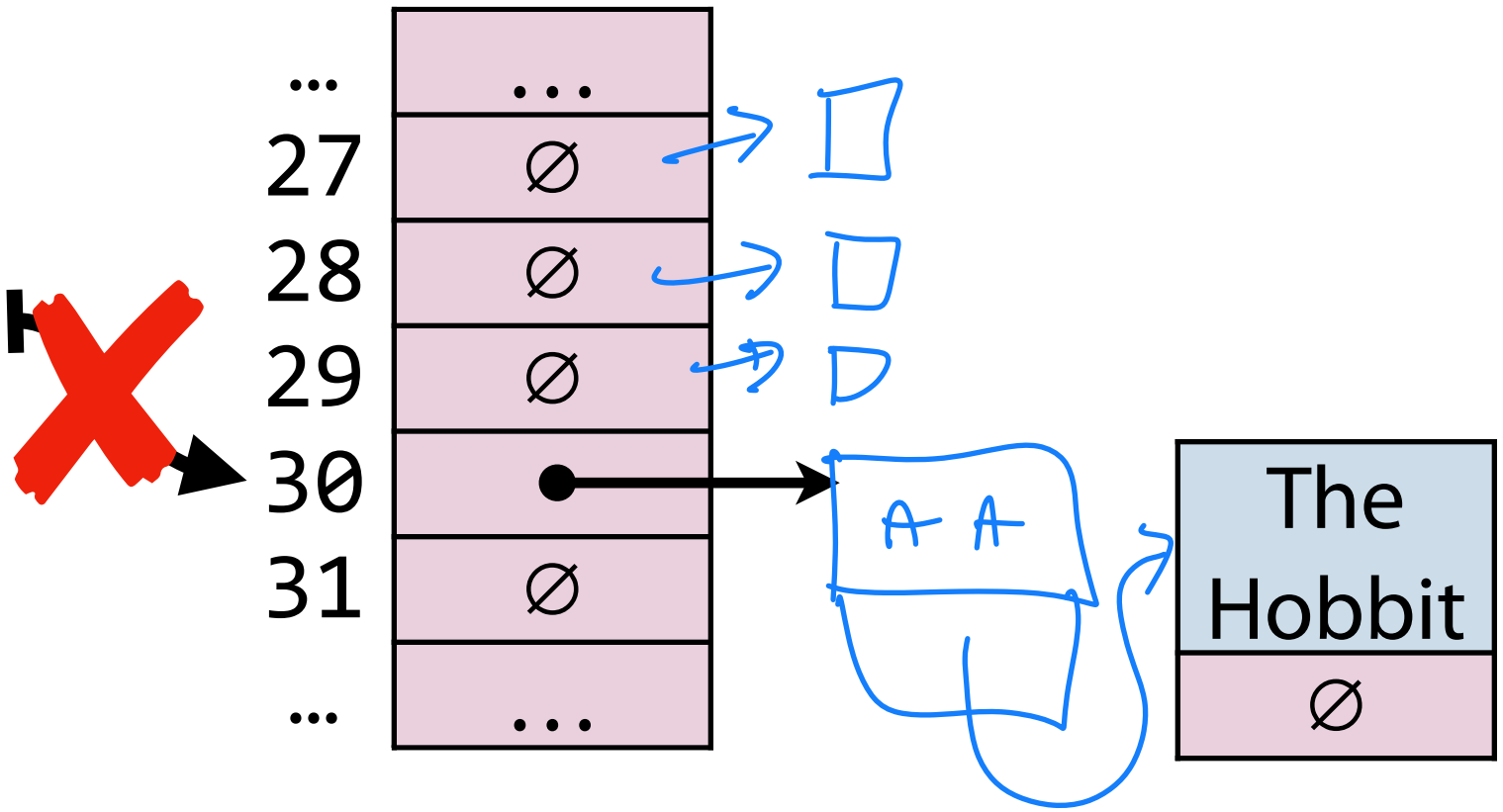




Hash Collisions (Open Hashing)

A ***hash collision*** in an open hashing scheme can be resolved by add to the linked list. This is called ***separate chaining***.

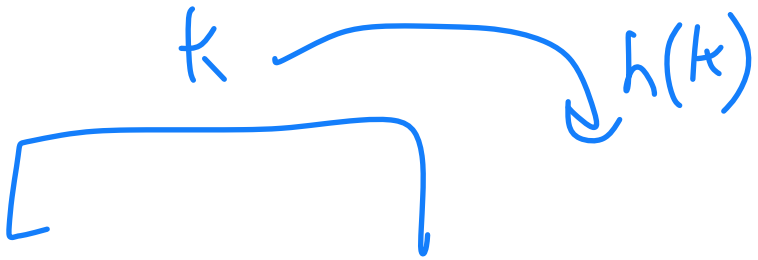




Insertion (Separate Chaining)

```
_insert("Bob")

_insert("Anna")
```

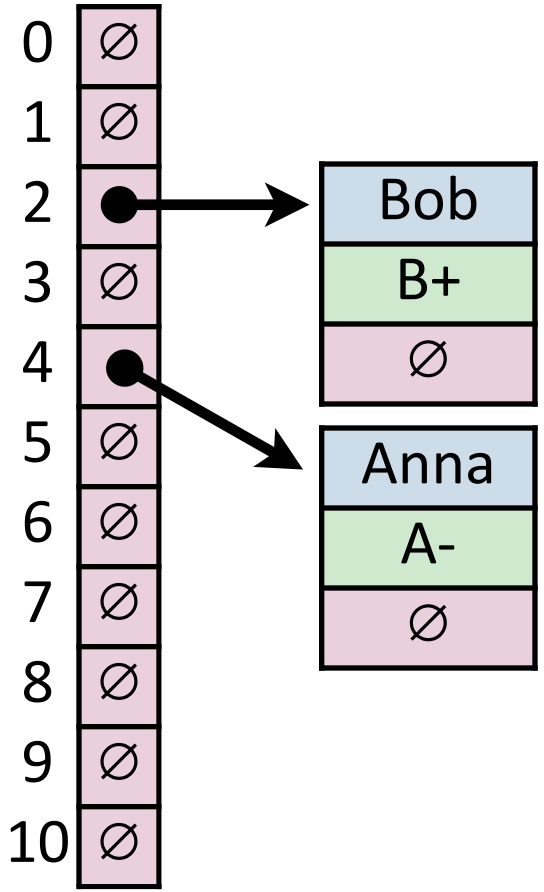


Key	Value	Hash
Bob	**B+**	**2**
Anna	**A-**	**4**
Alice	A+	4
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7

Insertion (Separate Chaining)

```
_insert("Alice")
```

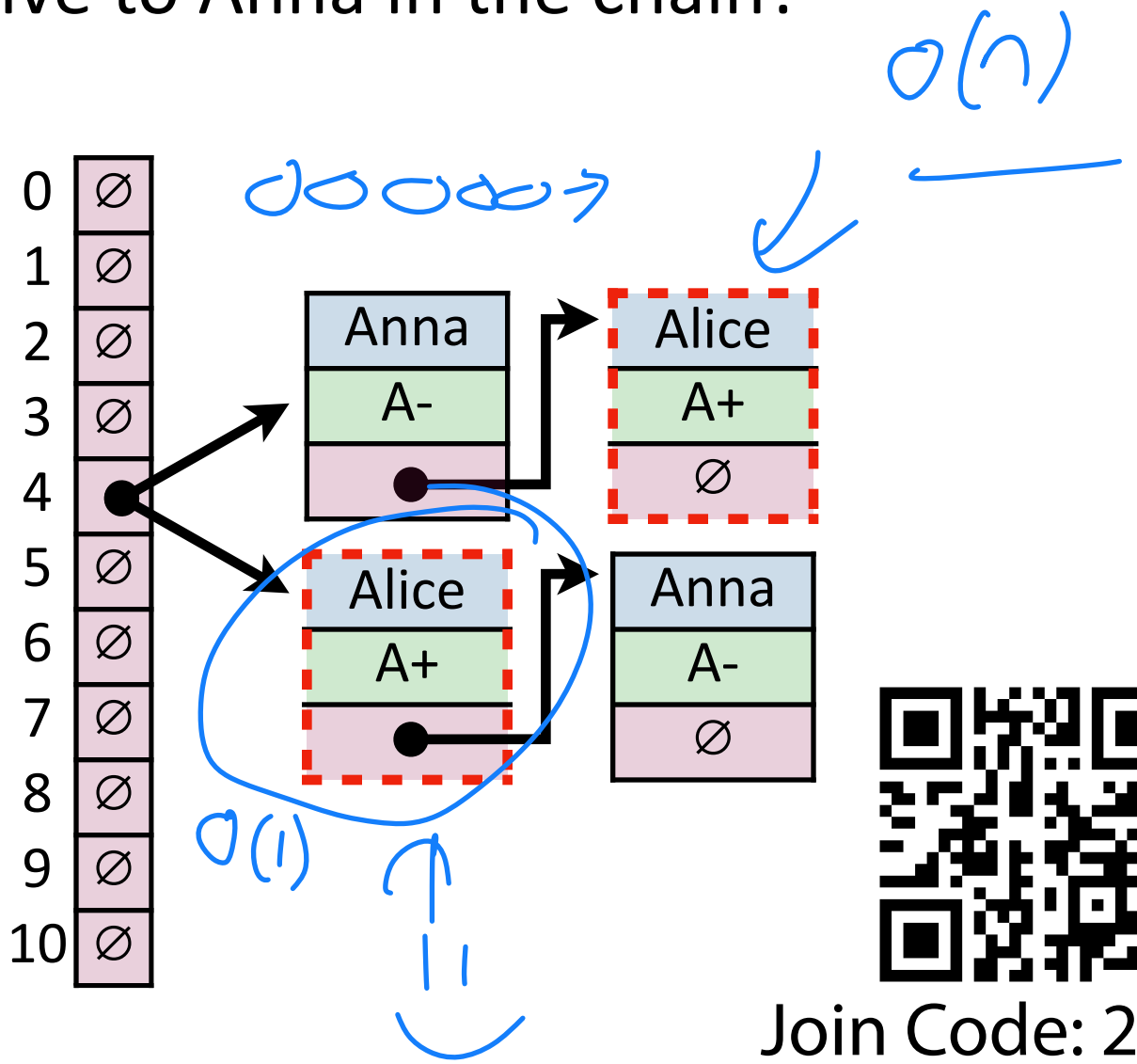
Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	**A+**	**4**
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Insertion (Separate Chaining)

Where does Alice end up relative to Anna in the chain?

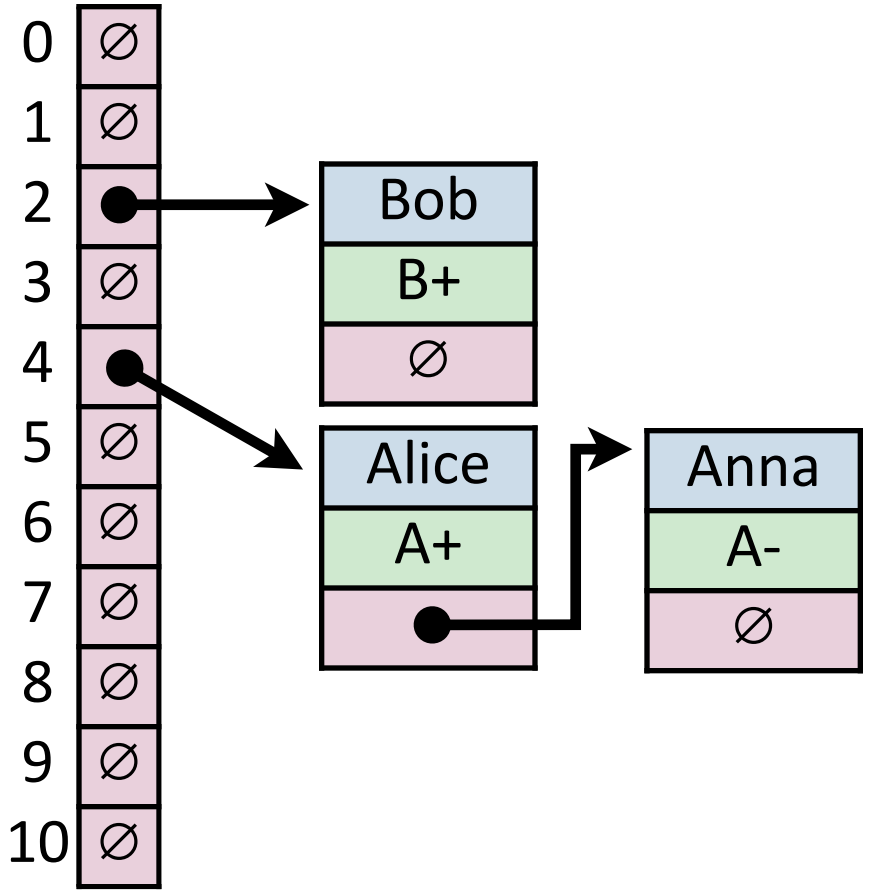
Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	**A+**	**4**
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Join Code: 277

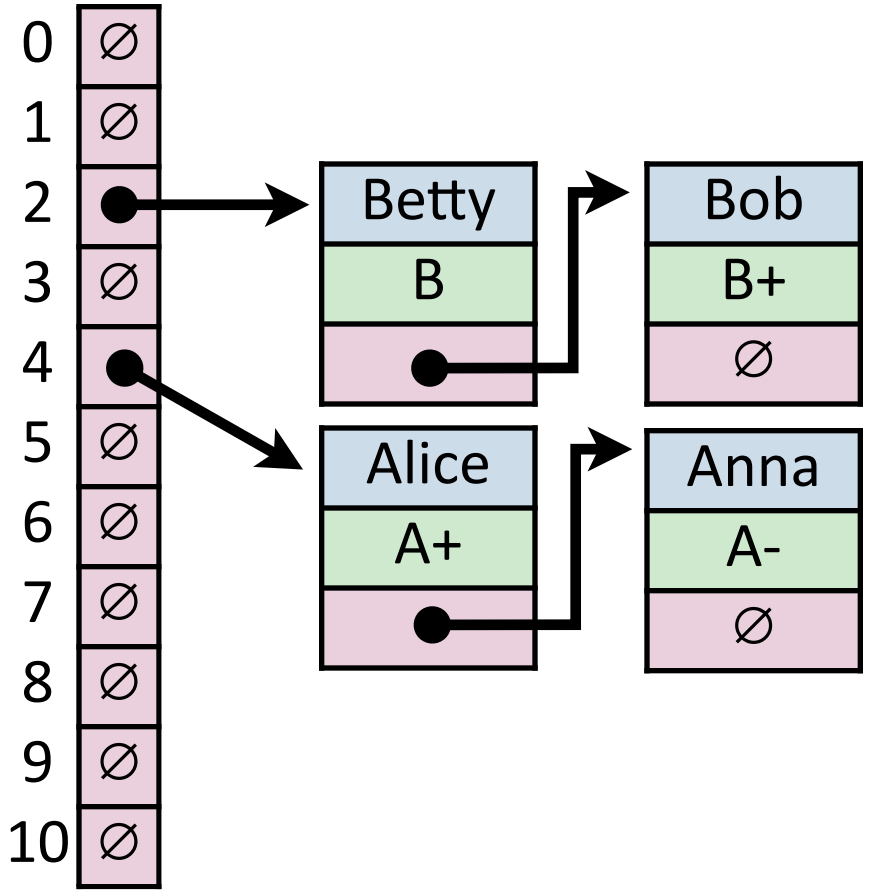
Insertion (Separate Chaining)

Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	A+	4
Betty	**B**	**2**
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Insertion (Separate Chaining)

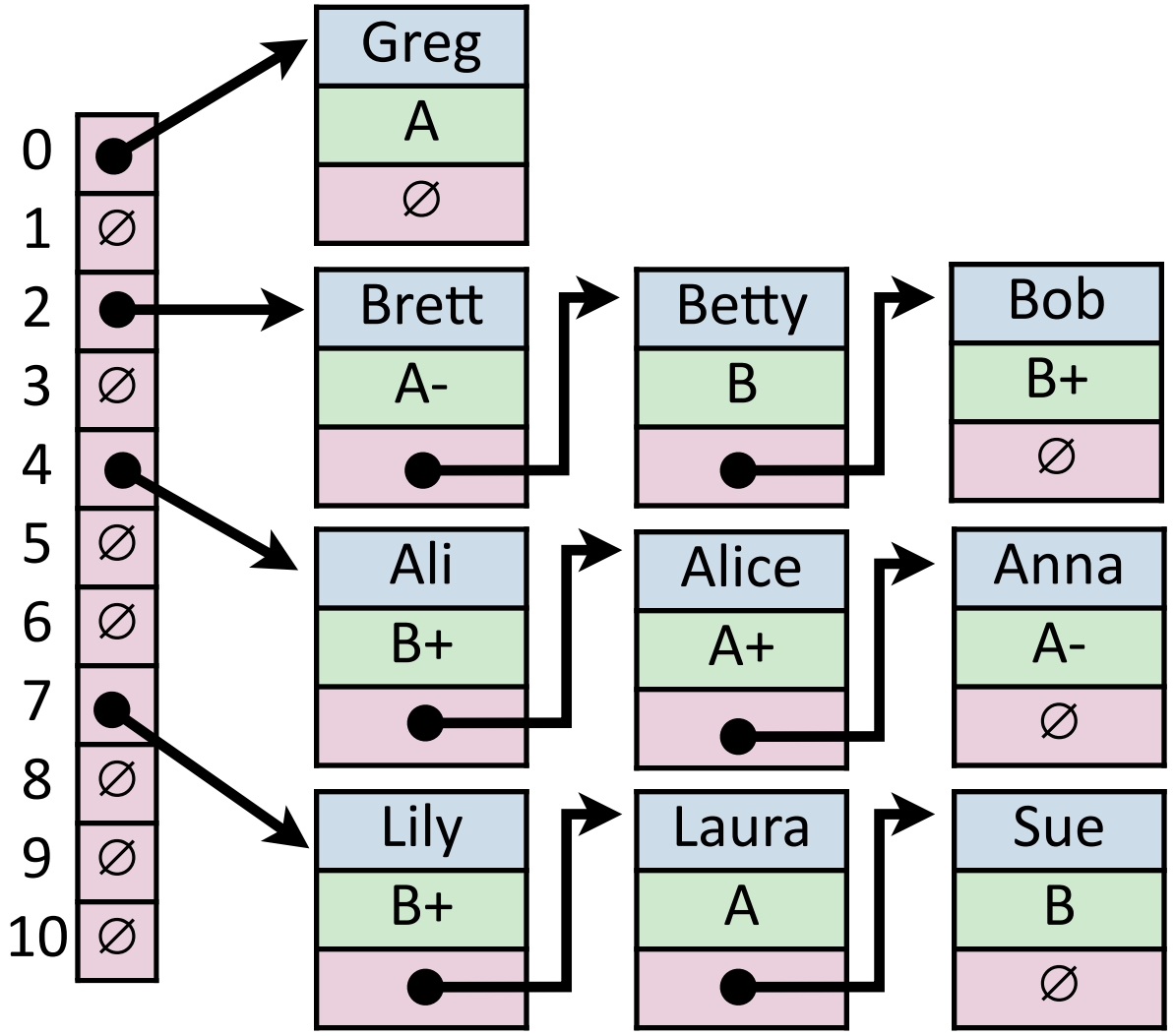
Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	A+	4
Betty	**B**	**2**
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Insertion (Separate Chaining)

An array of linked lists

Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	A+	4
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Find (Separate Chaining)

`_find("Sue")`

O(1) [Hash]

Key	Hash
Sue	7

A) O(1)

B) O(log n)

C) O(n)

An array

Start + size * i

Find ? in the array?

O(1)

[to lookup in hash table array]

0	∅
1	∅
2	∅
3	∅
4	∅
5	∅
6	∅
7	
8	∅
9	∅
10	∅

w/ one linked list at every pos'n

???

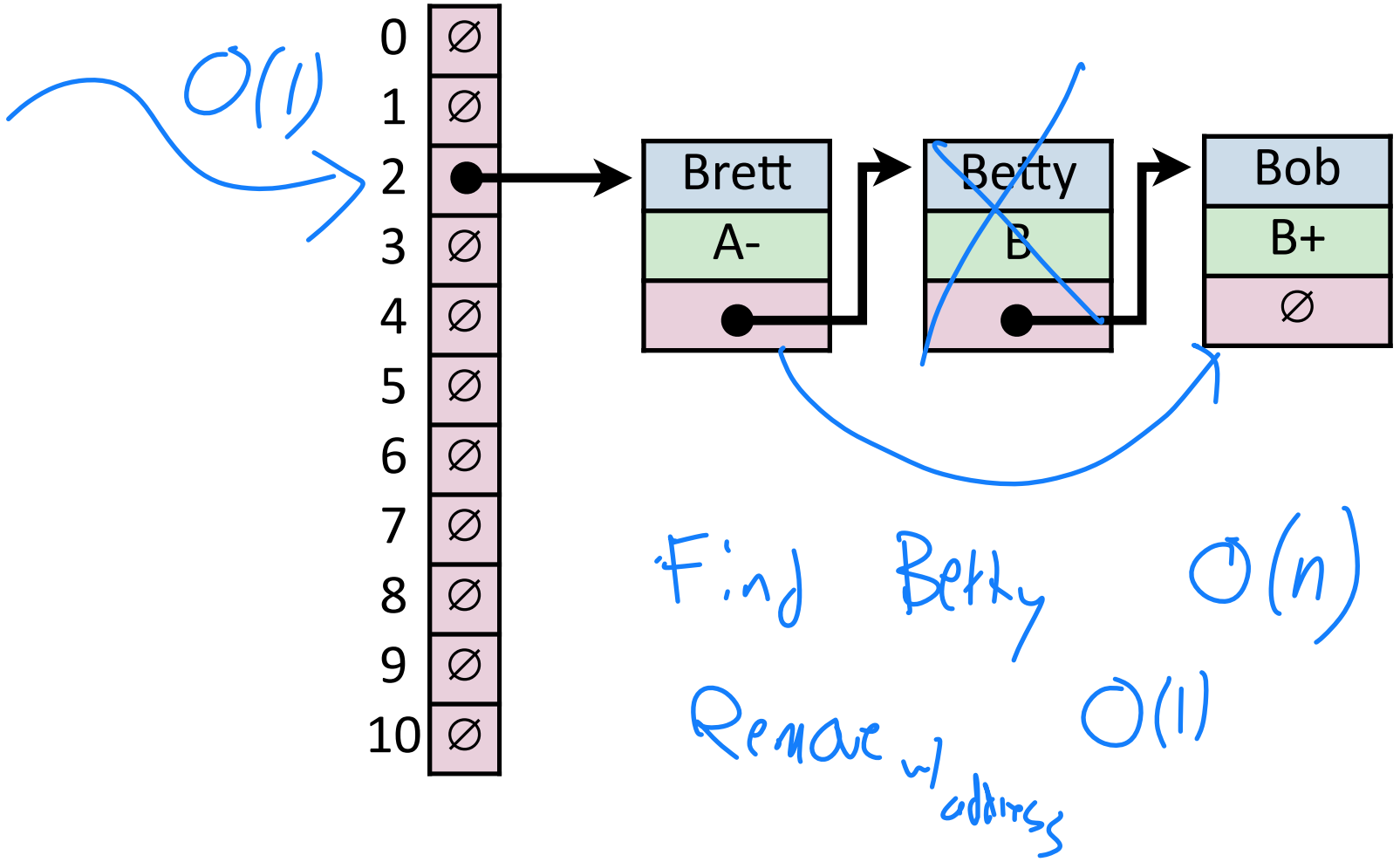
O(n) !!

Lily	Laura	Sue
B+	A	B
		∅

Remove (Separate Chaining)

`_remove("Betty")`

Key	Hash
Betty	2



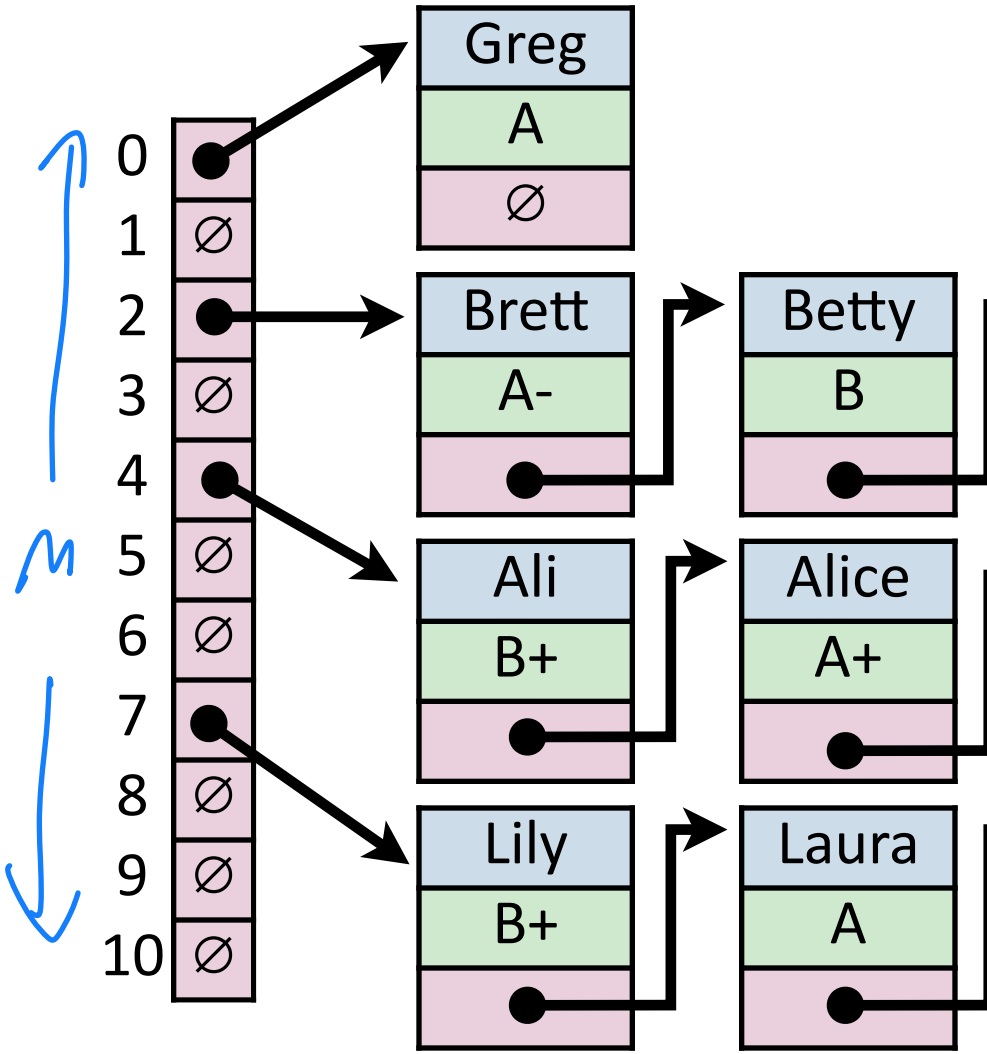
Hash Table (Separate Chaining)

For hash table of size *m* and *n* elements:

Find runs in: $O(1 + 1 + n) = O(n)$ (Hash, Lookup, Find in LL)

Insert runs in: $O(1 + 1 + 1) = O(1)$!!

Remove runs in: $O(n)$



Hash Table

Worst-Case behavior is bad — but how about in practice?

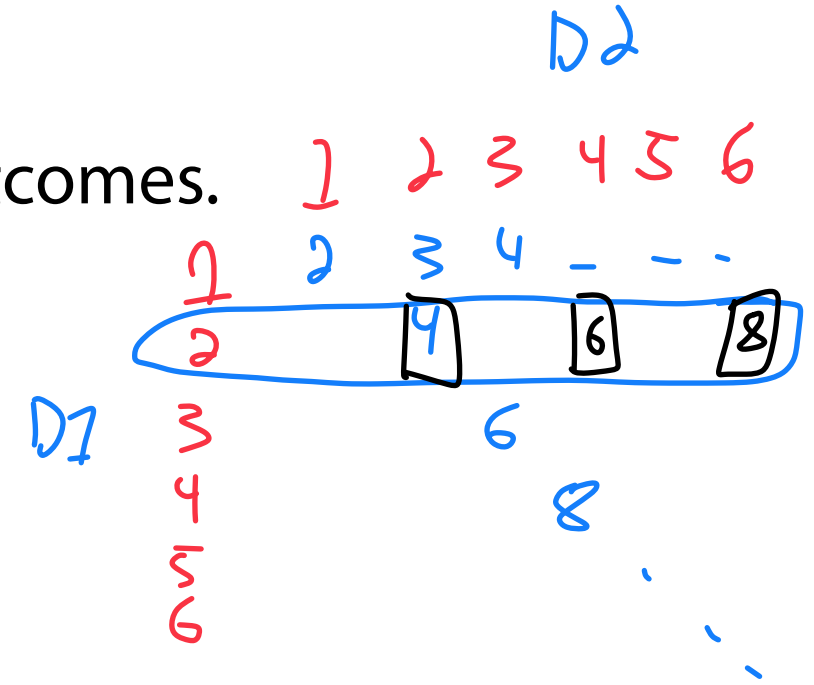
On average, a hash table (with a good hash) is constant time.

We will prove this mathematically using **probability!**

Fundamentals of Probability

Imagine you roll a pair of six-sided dice.

The **sample space** Ω is the set of all possible outcomes.



An **event** $E \subseteq \Omega$ is any subset of outcomes.

D1 rolls 2 and D2 is even ← Prob that this event occurs?

↳ $3/36$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

The **expectation** of a (discrete) random variable is:

$$E[X] = \sum_{x \in \Omega} Pr\{X = x\} \cdot x$$

Prob of event

value of event

$$\frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 2 + \frac{1}{6} \cdot 3 + \ldots \approx 3.5$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X + Y] = E[X] + E[Y]$$

$3.5 + 3.5 = 7$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X + Y] = E[X] + E[Y]$$

$$= \sum_x \sum_y Pr\{X = x, Y = y\}(x + y)$$

Prob of event Value of event

↑ ↑

Summing all possible events

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X+Y] = E[X] + E[Y]$$

$$= \sum_x \sum_y Pr\{X = x, Y = y\}(x + y)$$

$$= \sum_x x \sum_y Pr\{X = x, Y = y\} + \sum_y y \sum_x Pr\{X = x, Y = y\}$$

Sum of all probabilities is 1

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X+Y] = E[X] + E[Y]$$

$$= \sum_x \sum_y Pr\{X = x, Y = y\}(x+y)$$

$$= \sum_x x \sum_y Pr\{X = x, Y = y\} + \sum_y y \sum_x Pr\{X = x, Y = y\}$$

$$= \sum_x x \cdot Pr\{X = x\} + \sum_y y \cdot Pr\{Y = y\} = E[X] + E[Y]$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X + Y] = E[X] + E[Y]$$

Rolling two dice has expected value 7

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

# of collisions

α_j = expected # of items hashing to position j

Prob(item 0 hashes to j) + Prob(item 1 hashes to j)

Prob · value

Prob() · V

1

value is 1

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

↑
sum of all n items

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E\left[\sum_i H_{i,j}\right]$$

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$E[\alpha_j] = \sum_i Pr(H_{i,j} = 1) * 1 + Pr(H_{i,j} = 0) * 0$$

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E\left[\sum_i H_{i,j}\right]$$

But what is $Pr(H_{i,j} = 1)$?

$$E[\alpha_j] = \sum_i Pr(H_{i,j} = 1)$$

Simple Uniform Hashing Assumption

Given table of size m, a simple uniform hash, h, implies

$$\forall k_1, k_2 \in U \text{ where } k_1 \neq k_2, \; Pr(h[k_1] = h[k_2]) = \frac{1}{m}$$

Uniform: Every key equally likely to hash anywhere

Independent: All keys hash independent from each other

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$E[\alpha_j] = \sum_i Pr(H_{i,j} = 1)$$

But what is $Pr(H_{i,j} = 1)$?

SUHA says… $Pr[H_{i,j} = 1] = \frac{1}{m}$

Fundamentals of Probability (for Hash Tables)

Imagine you hashed n random items to an array of size m.

What is the expected number of items at a given position?

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$E[\alpha_j] = \sum_i \frac{1}{m} = \frac{n}{m}$$

Load factor in an array = n/m

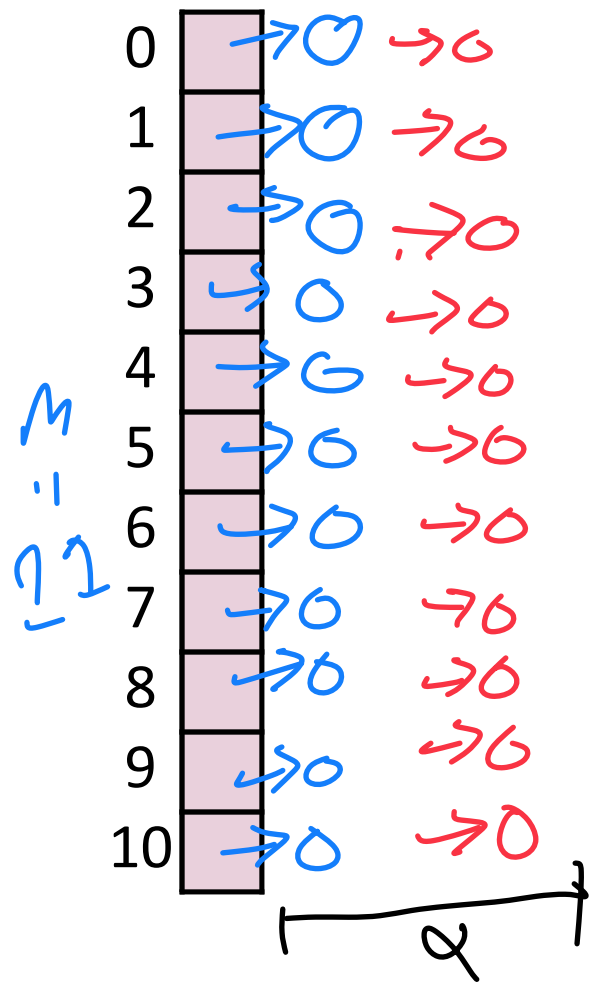
Separate Chaining Under SUHA

Under SUHA, a hash table of size *m* with *n* elements:

Find runs in: $1 + \alpha$.

Insert runs in: 1.

Remove runs in: $1 + \alpha$.



$\frac{n \text{ items}}{m \text{ positions}}$

$$\mathbf{E}[\alpha] = \frac{\mathbf{n}}{\mathbf{m}}$$

Open vs Closed Hashing

Addressing hash collisions depends on your storage structure.

- **Open Hashing:** store *k,v* pairs externally

↳ An array of linked lists

- **Closed Hashing:** store *k,v* pairs in the hash table

↳ An array

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 1, 8 , 15} ← keys

h(k) = k % 7

|S| = n

|Array| = m

place in "next available spot"

(Example of closed hashing)

Collision Handling: Linear Probing

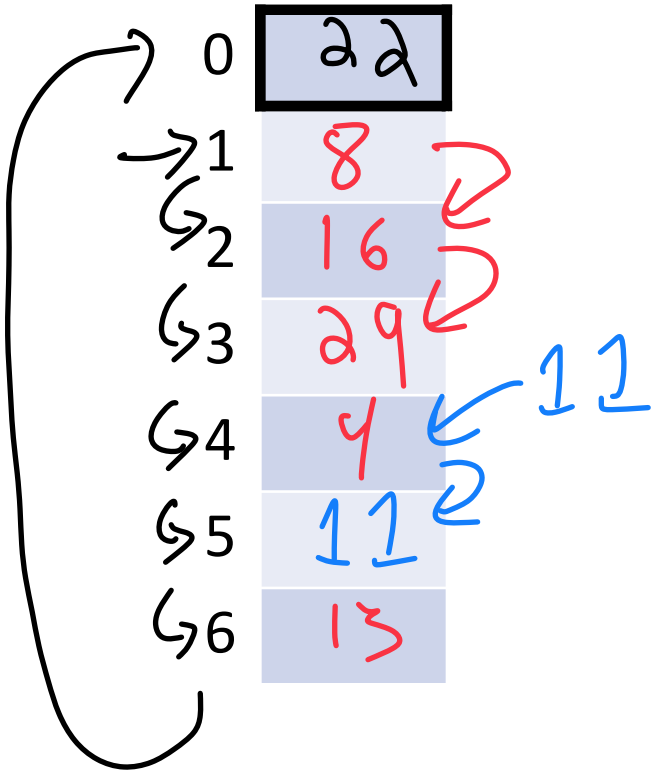
S = { 16, 8, 4, 13, 29, 11, 22 }

|S| = n

h(k) = k % 7

|Array| = m

What value is in pos 0?



h(k, i) = (k + i) % 7

Try h(k) = (k + 0) % 7, if full...

Try h(k) = (k + 1) % 7, if full...

Try h(k) = (k + 2) % 7, if full...

Try ...

29 % 7 = 1

11 % 7 = 4

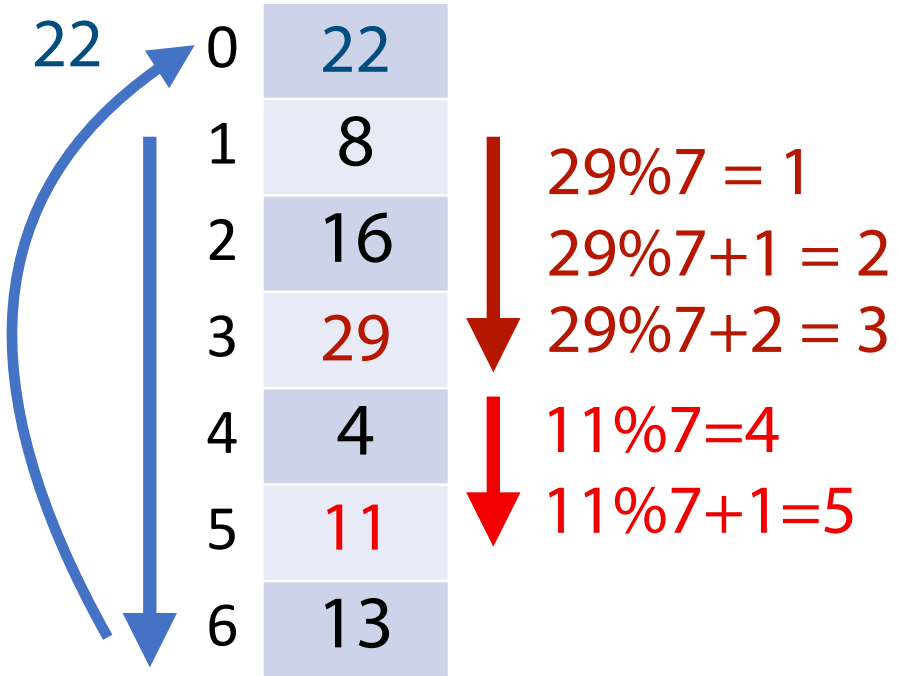
22 % 7 = 1

(Example of closed hashing)

Collision Handling: Linear Probing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = m**



22%7 = 1
22%7+1, +2, +3, +4, +5, +6

h(k, i) = (k + i) % 7

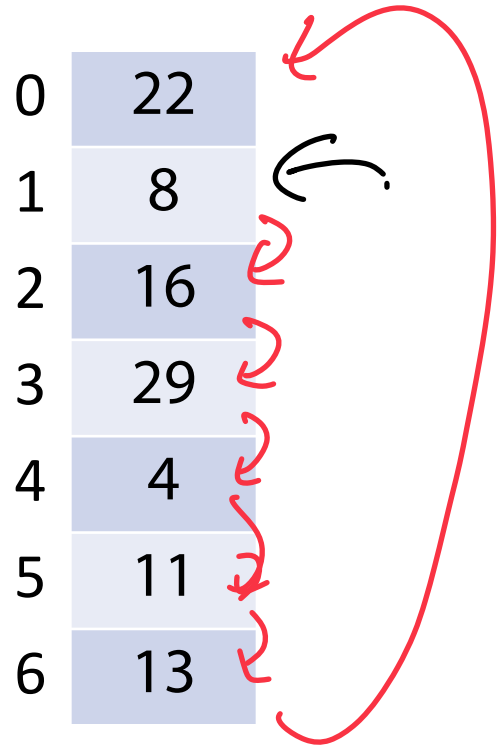
Try h(k) = (k + 0) % 7, if full…
Try h(k) = (k + 1) % 7, if full…
Try h(k) = (k + 2) % 7, if full…
Try …

(Example of closed hashing)

Collision Handling: Linear Probing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k, i) = (k + i) % 7 **|Array| = m**



`_find(29)`

1) Hash key 29 %7 = 2

2) Look up hash value (address)

3) Keep looking at "next available" until:

3.1) We find value!

3.2) Return to start pos

3.3) when we reach a blank space (that has no tombstone)

(Example of closed hashing)

Collision Handling: Linear Probing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k, i) = (k + i) % 7 **|Array| = m**

`_find(30)`

30 % 7 = 2

0	22
1	8
2	16
3	29
4	
5	11
6	13

Stop Search!

If 30 in table, would be here!

How many steps does it take to find 30 (or see that its not there?)

(Example of closed hashing)

Collision Handling: Linear Probing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k, i) = (k + i) % 7 **|Array| = m**

0	22
1	8
2	~~16~~
3	29
4	4
5	11
6	13

tombstone = 1
↳ Something was here (and isn't anymore)

`_remove(16)`

16 % 7 = 2

1) Hash
2) Find (look until it's found or we hit blank or we saw everything)
3) Remove value
No moving other values!
4) Tombstone space

(Example of closed hashing)

Collision Handling: Linear Probing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k, i) = (k + i) % 7 **|Array| = m**

```
_remove(16)
_find(29)
```

0	22
1	8
2	
3	29
4	4
5	11
6	13

Can store at most n items

Collision Handling: Probe-Based Hashing

A fixed array where a single (k, v) pair can exist in a cell

When a collision occurs, look at the **next available space**

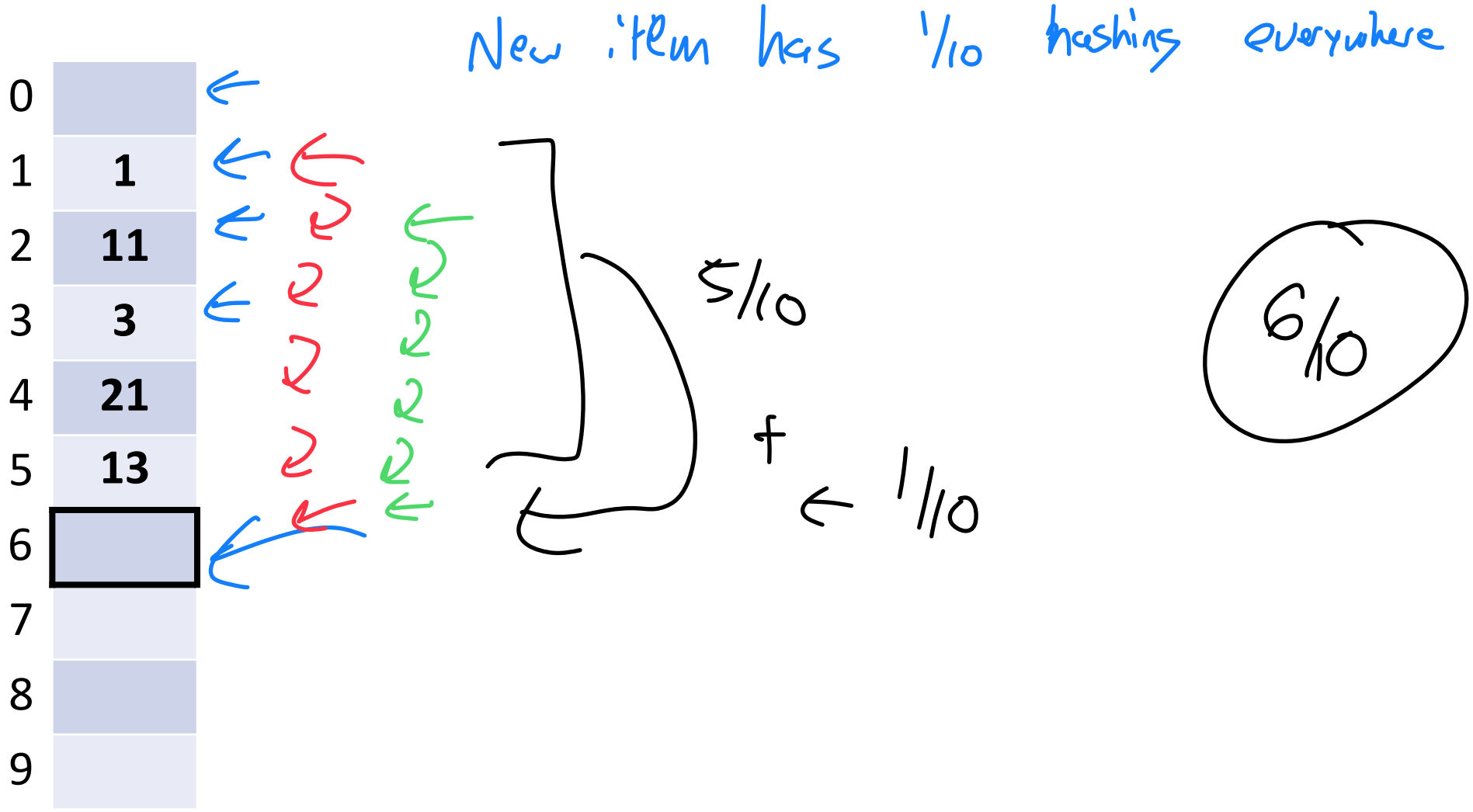
When removing from the table, **tombstone** to preserve ability to find

When finding, continue searching until:

- Key is found
- A blank cell (with no tombstone) is found
- All positions have been searched

A Problem w/ Linear Probing

What is the probability that the next item hashes to the black square (@6)?



A Problem w/ Linear Probing

Primary Clustering:

0	
1	1_1
2	1_2
3	3_1
4	1_3
5	3_2
6	
7	
8	
9	

Description: Long clusters of (k,v) pairs grow

I'm more likely to grow a large chain

Solution: Don't use linear probing!

Better "next available"

(Example of closed hashing)

Collision Handling: Quadratic Probing

S = { 16, 8, 4, 13, 29, 12, 22 }

|S| = n

h(k) = k % 7

|Array| = m

0	12
1	8
2	16
3	22
4	4
5	29
6	13

h(k, i) = (k + i*i) % 7

Try h(k) = (k + 0) % 7, if full...

Try h(k) = (k + 1*1) % 7, if full...

Try h(k) = (k + 2*2) % 7, if full...

Try ...

29 % 7 = 1

1 + 1*1

1 + 2*2 = 5

12 % 7 = 5

5 + 1 = 6

5 + 4 = 9 % 7 = 2

5 + 9 = 14 % 7 = 0

A Problem w/ Quadratic Probing

Where does the next item with a remainder of 0 go?

h(x) % 10

40

40 % 10 = 0

0 + 1 * 1

0 + 2 * 2

0 + 3 * 3 = 9

0	**10**
1	**20**
2	
3	
4	**30**
5	
6	
7	
8	
9	40

M = 10

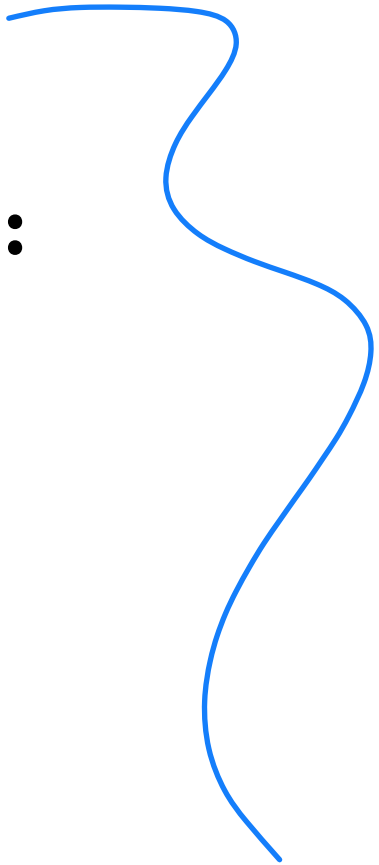
A Problem w/ Quadratic Probing

Secondary Clustering:

0	O_1
1	O_2
2	
3	
4	O_3
5	
6	
7	
8	
9	O_4

Description:

Remedy:



Stopped here!

(Example of closed hashing)

Collision Handling: Double Hashing

S = { 16, 8, 4, 13, 29, 11, 22 }

$h_1(k) = k \% 7$

$h_2(k) = 5 - (k \% 5)$

|S| = n

|Array| = m

0	
1	**8**
2	**16**
3	
4	**4**
5	
6	**13**

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

Running Times

(Expectation under SUHA)

(Understand why we have these rough forms)

Open Hashing:

insert: _____.

find/ remove: _____.

Closed Hashing:

insert: _____.

find/ remove: _____.

Running Times (Expectation under SUHA)

Open Hashing: $0 \leq \alpha \leq \infty$

insert: 1.

find/ remove: $1 + \alpha$.

Closed Hashing: $0 \leq \alpha < 1$

insert: $\frac{1}{1-\alpha}$.

find/ remove: $\frac{1}{1-\alpha}$.

Observe:

- As α increases:

- If α is constant:

Running Times

The expected number of probes for find(key) under SUHA

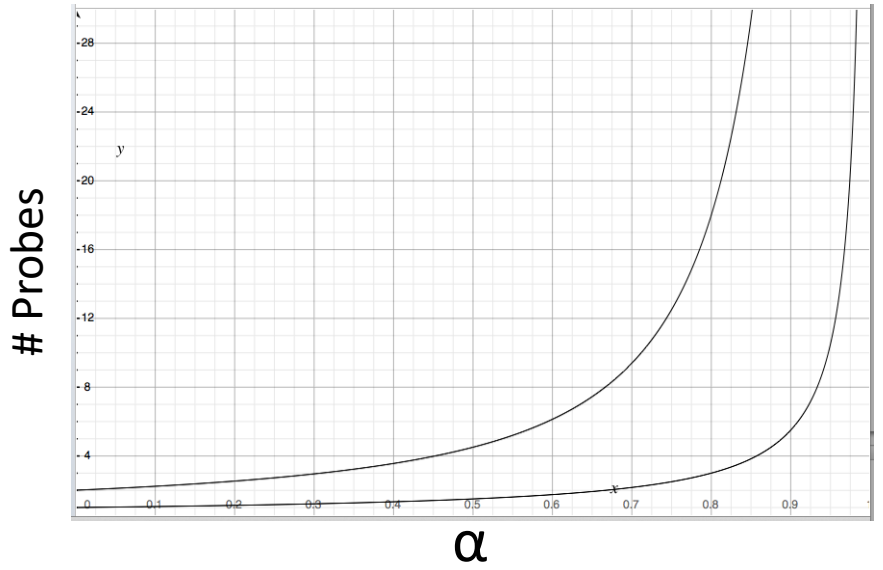
Linear Probing:

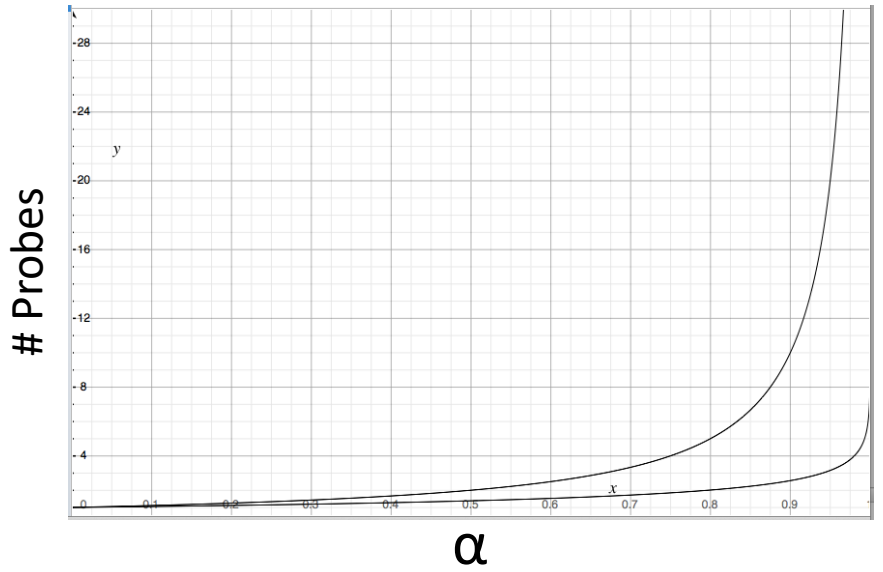
- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**

Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

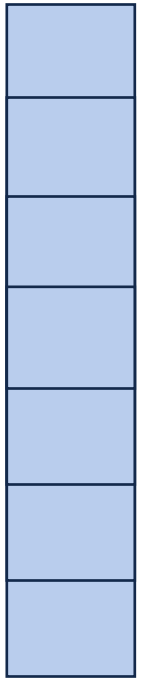
When do we resize?

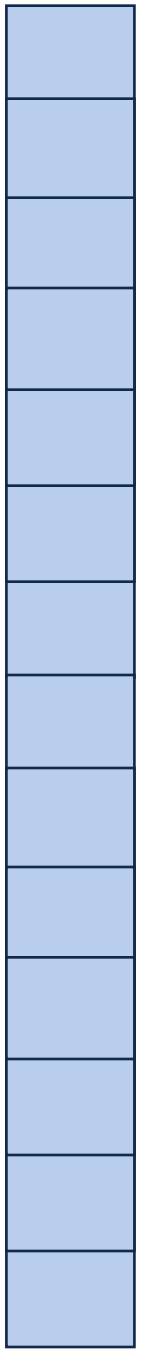




Resizing a hash table

How do you resize?





Running Times Review

	Hash Table	Array List	Linked List
Find	Expectation*: Worst Case:		
Insert	Expectation*: Worst Case:		
Remove	Expectation*: Worst Case:		
Storage Space			

Bonus Slides

Choosing a Hash Function

Python has a built-in hash! It's pretty good if you run everything at once.

```
1 print(hash("I can pass in any string!”))
2
3
4 print(hash(205811))
5
6
```

Choosing a Hash Function

If you want something that is persistently deterministic, find a seeded hash

```
1 import mmh3
2
3 print(mmh3.hash("I can pass in any string!", 10)) #I got: -565691678
4 print(mmh3.hash("I can pass in any string!", 50)) #I got: -947521776
5 print(mmh3.hash("I can pass in any string!", 12)) #I got: 1680496801
6
```

Hash Table

SUHA is an assumption… is it a realistic one?

Hash Table

SUHA is an assumption… is it a realistic one?

Yes — it’s actually possible to build a SUHA hash function!

One way is to create a ***universal hash function family***

Hash Function (Division Method)

Hash of form: $h(k) = k \,\%\, m$

Pro:

Con:

Hash Function (Multiplication Method)

Hash of form: $h(k) = \lfloor m(kA \,\%\, 1) \rfloor,\ 0 \leq A \leq 1$

Pro:

Con:

Hash Function (Universal Hash Family)

Hash of form: $h_{ab}(k) = \big((ak + b)\,\%\,p\big)\,\%\,m,\ a, b \in Z_p^*, Z_p$

$$\forall k_1 \neq k_2,\ Pr_{a,b}(h_{ab}[k_1] = h_{ab}[k_2]) \leq \frac{1}{m}$$

Pro:

Con: